REVISION CHEAT SHEET:

EVENTS / C#

Events and Observer-Style Programming

Interview-ready notes · examples · C# · diagrams

Topic 23 of 40

Events let one object announce:

> **“Something happened.”**

without requiring that object to know exactly who is interested in the notification.

This creates three important roles:

EXAMPLE / FLOW

```
Publisher
    ↓
Raises event
    ↓
Subscriber(s)
    ↓
React to event
```

For the Fundraising Platform:

EXAMPLE / FLOW

```
Gift processing completes
        ↓
GiftCompleted event
        ↓
 ┌──────────────┬──────────────┬──────────────┐
 ↓              ↓              ↓
Receipt      Audit         Analytics
handler      handler        handler
```

The publisher doesn't need direct dependencies on every subscriber.

C# events are based on delegates. An event has a delegate type, subscribers add compatible handlers, and when the publisher raises the event, the subscribed handlers are invoked. Events can have multiple subscribers, and ordinary event invocation is synchronous.

1. Why Events Exist

Suppose `GiftService` directly performs everything:

C#

```
public void CompleteGift(Gift gift)
{
    gift.Complete();

    SendReceipt(gift);
    WriteAuditLog(gift);
    UpdateDashboard(gift);
    NotifyDevelopmentOfficer(gift);
}
```

Now `GiftService` knows about:

EXAMPLE / FLOW

```
Receipt delivery

Audit logging

Dashboard updates

Notifications
```


The more reactions you add, the more responsibilities become coupled together.

An event-based design can instead say:

C#

```
GiftCompleted?.Invoke(
    this,
    eventArgs);
```

Subscribers independently decide what to do.

2. Publisher and Subscriber

The object declaring and raising an event is usually called the:

EXAMPLE / FLOW

```
Publisher
```

The object attaching a handler is the:

EXAMPLE / FLOW

```
Subscriber
```

Example publisher:

C#

```
public sealed class GiftProcessor
{
    public event EventHandler?
        GiftProcessed;

    public void Process()
    {
        // Work...

        GiftProcessed?.Invoke(
            this,
            EventArgs.Empty);
    }
}
```

Subscriber:

C#

```
public sealed class AuditSubscriber
{
    public void Subscribe(
        GiftProcessor processor)
    {
        processor.GiftProcessed += HandleGiftProcessed;
```

```
    }

    private void HandleGiftProcessed(
        object? sender,
        EventArgs e)
    {
        Console.WriteLine(
            "Gift processed.");
    }
}
```

3. The event Keyword

An event declaration looks like:

C#
```
public event EventHandler?
    GiftProcessed;
```

The:

C#
```
event
```

keyword gives the delegate special encapsulation semantics.

External code can normally:

EXAMPLE / FLOW
```
Subscribe

Unsubscribe
```

but cannot directly raise the event.

Microsoft describes an event as a member that allows an object to trigger notifications, while event consumers attach or remove handlers.

4. Event vs Plain Delegate

Consider exposing a plain delegate:

C#
```
public Action<Gift>?
    GiftCompleted;
```

External code could potentially do:

C#
```
processor.GiftCompleted = MyHandler;
```

replacing other subscribers.

It could also invoke it:

C#
```
processor.GiftCompleted?.Invoke(
    gift);
```

That gives consumers too much control.

Event Version

C#
```
public event Action<Gift>?
    GiftCompleted;
```

External consumers can do:

C#
```
processor.GiftCompleted += MyHandler;
```

and:

C#
```
processor.GiftCompleted -= MyHandler;
```

but they cannot normally do:

C#
```
processor.GiftCompleted = MyHandler;
```

or raise it themselves.

This is the key difference:

EXAMPLE / FLOW
```
Delegate
    →
Callable behavior object

Event
    →
Controlled publisher/subscriber abstraction
built on a delegate
```

5. Basic Event Example

Publisher:

C#
```
public sealed class GiftProcessor
{
    public event Action<Gift>?
        GiftCompleted;

    public void Complete(
        Gift gift)
    {
        ArgumentNullException.ThrowIfNull(
            gift);

        gift.Complete();

        GiftCompleted?.Invoke(
            gift);
    }
}
```

Subscriber:

```C#
public sealed class GiftLogger
{
    public void Handle(
        Gift gift)
    {
        Console.WriteLine(
            $"Completed Gift {gift.GiftId}");
    }
}
```

Subscription:

```C#
GiftProcessor processor = new();

GiftLogger logger = new();

processor.GiftCompleted += logger.Handle;
```

Runtime Flow

EXAMPLE / FLOW

```
processor.Complete(gift)
        ↓
Gift completes
        ↓
GiftCompleted raised
        ↓
logger.Handle(gift)
```

6. Subscribe with +=

Subscription uses:

```C#
+=
```

Example:

```C#
processor.GiftCompleted += logger.Handle;
```

Multiple subscribers:

```C#
processor.GiftCompleted += logger.Handle;

processor.GiftCompleted += receiptSender.Handle;

processor.GiftCompleted += analytics.Handle;
```

The event's underlying multicast delegate maintains these subscribed handlers.

7. Unsubscribe with -=

Use:

```C#
-=
```

Example:

```C#
processor.GiftCompleted -= logger.Handle;
```

After unsubscribing, that handler is no longer part of future event notification.

8. Why Unsubscription Matters

Unsubscription isn't only about stopping notifications.

It can also affect object lifetime.

A publisher's event delegate holds references to subscribed handlers. If the publisher lives longer than the subscriber, that reference can keep the subscriber reachable and prevent garbage collection. Microsoft specifically recommends unsubscribing when necessary to avoid this resource/lifetime problem.

Conceptually:

EXAMPLE / FLOW

```
Long-lived Publisher
        ↓
Event delegate
        ↓
Subscriber handler
        ↓
Subscriber object
```

As long as that chain remains reachable:

EXAMPLE / FLOW

```
Subscriber
may remain alive
```

9. Event Subscription Lifetime Rule

The important question is:

> **Which object lives longer: publisher or subscriber?**

If both have the same short lifetime:

EXAMPLE / FLOW

```
Request-scoped Publisher
+
Request-scoped Subscriber
```

the issue may disappear naturally when the whole object graph becomes unreachable.

But if:

EXAMPLE / FLOW

```
Singleton Publisher
        ↓
Scoped/Transient Subscriber
```

then an event subscription can accidentally extend the subscriber's lifetime.

10. Event Memory-Leak Example

C#

```
public sealed class GiftMonitor
{
    public event EventHandler?
        GiftChanged;
}
```

Long-lived:

C#

```
GiftMonitor monitor = applicationSingleton;
```

Short-lived subscriber:

C#

```
public sealed class GiftPage :
    IDisposable
{
    private readonly GiftMonitor
        _monitor;

    public GiftPage(
        GiftMonitor monitor)
    {
        _monitor = monitor;

        _monitor.GiftChanged += HandleGiftChanged;
    }

    private void HandleGiftChanged(
        object? sender,
        EventArgs e)
    {
    }

    public void Dispose()
    {
        _monitor.GiftChanged -= HandleGiftChanged;
    }
}
```

Unsubscription breaks:

EXAMPLE / FLOW

```
Publisher → Subscriber
```

when the subscriber is finished.

11. Lambda Unsubscription Problem

EVENTS / C#

This is easy:

```
C#
publisher.GiftCompleted += HandleGiftCompleted;
```

because later you can write:

```
C#
publisher.GiftCompleted -= HandleGiftCompleted;
```

But this is more difficult:

```
C#
publisher.GiftCompleted += gift => Console.WriteLine(gift.GiftId);
```

You don't automatically have the same delegate instance available later for removal.

Microsoft recommends storing an anonymous handler in a variable if it must later be unsubscribed.

Better

```
C#
Action<Gift> handler = gift => Console.WriteLine(gift.GiftId);

publisher.GiftCompleted += handler;
```

Later:

```
C#
publisher.GiftCompleted -= handler;
```

12. Raising an Event

For a field-like event:

```
C#
public event EventHandler?
    GiftProcessed;
```

the publisher commonly raises it with:

```
C#
GiftProcessed?.Invoke(
    this,
    EventArgs.Empty);
```

The null-conditional invocation means:

```
EXAMPLE / FLOW
No subscribers
    →
Do nothing
```

rather than attempting to invoke `null`.

Microsoft's current event example uses this pattern for raising an event.

13. Why ?.Invoke

Instead of:

```C#
if (
    GiftProcessed != null)
{
    GiftProcessed(
        this,
        EventArgs.Empty);
}
```

modern C# commonly uses:

```C#
GiftProcessed?.Invoke(
    this,
    EventArgs.Empty);
```

It is:

```EXAMPLE / FLOW
Concise

Clear

Safe against the event being null
```

It does **not** mean the entire publisher or every handler is magically thread-safe.

14. Standard .NET Event Pattern

Although events can use any delegate type, conventional reusable .NET APIs normally use:

```C#
EventHandler
```

or:

```C#
EventHandler<TEventArgs>
```

The standard handler shape is:

```EXAMPLE / FLOW
sender
+
event data
```

Microsoft's standard event pattern uses `object` for the sender and an `EventArgs`-based value for event data.

15. EventHandler

Use:

```
C#
EventHandler
```

when no custom event data is required.

Example:

```
C#
public event EventHandler?
    ProcessingStarted;
```

Raise:

```
C#
ProcessingStarted?.Invoke(
    this,
    EventArgs.Empty);
```

Handler:

```
C#
private void HandleProcessingStarted(
    object? sender,
    EventArgs e)
{
}
```

EventHandler Mental Model

```
EXAMPLE / FLOW
Publisher
    ↓
sender
    +
EventArgs
```

For no additional event data:

```
C#
EventArgs.Empty
```

is the standard value.

16. EventHandler<TEventArgs>

When subscribers need data:

```
C#
public event EventHandler<GiftCompletedEventArgs>?
    GiftCompleted;
```

Handler:

```
C#
private void HandleGiftCompleted(
    object? sender,
    GiftCompletedEventArgs e)
```

```
{
}
```

EVENTS / C#

This is the common .NET event pattern for events carrying data.

17. Custom EventArgs

Create an event data class:

C#

```
public sealed class GiftCompletedEventArgs :
    EventArgs
{
    public GiftCompletedEventArgs(
        long giftId,
        decimal amount,
        DateTimeOffset completedAt)
    {
        GiftId = giftId;

        Amount = amount;

        CompletedAt = completedAt;
    }

    public long GiftId
    {
        get;
    }

    public decimal Amount
    {
        get;
    }

    public DateTimeOffset CompletedAt
    {
        get;
    }
}
```

Microsoft's event pattern recommends deriving custom event-data types from `EventArgs` and naming them with the `EventArgs` suffix.

Raise with Data

C#

```
GiftCompleted?.Invoke(
    this,
    new GiftCompletedEventArgs(
        gift.GiftId,
        gift.TotalAmount,
        DateTimeOffset.UtcNow));
```

Subscriber receives:

EXAMPLE / FLOW

```
GiftId

Amount
```

```
CompletedAt
```


without querying the publisher again.

18. What Is sender?

In:

```
C#
void Handler(
    object? sender,
    GiftCompletedEventArgs e)
```

`sender` usually represents:

```
EXAMPLE / FLOW
The object that raised the event
```

For:

```
C#
GiftCompleted?.Invoke(
    this,
    eventArgs);
```

`sender` becomes:

```
C#
this
```

the current publisher.

Handler Example

```
C#
private void HandleGiftCompleted(
    object? sender,
    GiftCompletedEventArgs e)
{
    if (
        sender is GiftProcessor processor)
    {
        Console.WriteLine(
            $"Gift {e.GiftId} completed.");
    }
}
```

Often you don't need `sender`, because `e` contains the useful information.

19. Event Naming

The .NET naming guidelines recommend:

```
EXAMPLE / FLOW
Verb or verb phrase
```

and commonly use tense to distinguish:

EXAMPLE / FLOW

```
Something happening
vs
Something completed
```

EVENTS / C#

Examples:

EXAMPLE / FLOW

```
Closing
Closed

Processing
Processed

Completing
Completed
```

rather than names such as:

EXAMPLE / FLOW

```
BeforeCompleted

AfterCompleted
```

Custom event-data classes conventionally end in:

EXAMPLE / FLOW

```
EventArgs
```

Fundraising Naming

Good:

C#

```
GiftCompleting

GiftCompleted

AllocationAdded

CampaignClosed
```

Less clear:

C#

```
OnGift

GiftEvent1

AfterGift

GiftHandlerEvent
```

20. The `OnEventName` Pattern

A common reusable-class pattern is:

```
C#
public event EventHandler<GiftCompletedEventArgs>?
    GiftCompleted;

protected virtual void OnGiftCompleted(
    GiftCompletedEventArgs e)
{
    GiftCompleted?.Invoke(
        this,
        e);
}
```

Then business logic calls:

```
C#
OnGiftCompleted(
    eventArgs);
```

instead of invoking the event field everywhere.

Why This Helps

It centralizes event raising.

You now have:

```
EXAMPLE / FLOW
Business operation
    ↓
OnGiftCompleted(...)
    ↓
GiftCompleted?.Invoke(...)
```

Benefits include:

```
EXAMPLE / FLOW
One raising location

Inheritance hook

Easier customization

Consistent behavior
```

Microsoft recommends the protected event-raising method pattern when derived classes need to participate in event raising.

21. Derived Classes Cannot Directly Raise a Base Event

Suppose:

```
C#
public abstract class GiftProcessorBase
{
    public event EventHandler?
        Processed;
}
```

A derived class cannot simply invoke the base event directly as though it were its own delegate field.

Instead provide:

```C#
protected virtual void OnProcessed()
{
    Processed?.Invoke(
        this,
        EventArgs.Empty);
}
```

Derived:

```C#
public sealed class OnlineGiftProcessor :
    GiftProcessorBase
{
    public void Process()
    {
        // Work...

        OnProcessed();
    }
}
```

Microsoft's guidance specifically uses a protected invocation method for this scenario.

22. Avoid Virtual Events as the Normal Extension Mechanism

C# permits constructs such as:

```C#
public virtual event EventHandler?
    Changed;
```

but Microsoft recommends using:

EXAMPLE / FLOW

```
Non-virtual event
+
protected virtual OnChanged(...)
```

for base-class extensibility because overriding the event itself can create confusing subscription behavior.

Prefer:

```C#
public event EventHandler?
    Changed;

protected virtual void OnChanged()
{
    Changed?.Invoke(
        this,
        EventArgs.Empty);
}
```

23. Multiple Subscribers

An event may have:

```
0

1

Many
```

subscribers.

Example:

C#

```
processor.GiftCompleted += audit.Handle;

processor.GiftCompleted += receipt.Handle;

processor.GiftCompleted += analytics.Handle;
```

On raise:

EXAMPLE / FLOW

```
GiftCompleted
    ↓
Audit
    ↓
Receipt
    ↓
Analytics
```

Ordinary events use multicast delegate behavior, and subscribers execute synchronously during event raising.

24. Subscriber Exception Behavior

Suppose:

EXAMPLE / FLOW

```
Audit
    ↓
Receipt
    ↓
Analytics
```

and:

EXAMPLE / FLOW

```
Receipt throws
```

During normal multicast invocation:

EXAMPLE / FLOW

```
Audit executes

Receipt throws

Analytics is not reached

Exception propagates
```

unless the publisher or handler deliberately catches the exception.

This means publisher design must consider:

EXAMPLE / FLOW

```
Are subscriber failures allowed
to fail the publisher operation?
```

Don't Blindly Swallow Subscriber Exceptions

Bad:

C#

```
try
{
    GiftCompleted?.Invoke(
        this,
        args);
}
catch
{
}
```

Now:

EXAMPLE / FLOW

```
Subscribers can fail silently
```

which can hide serious problems.

If Handlers Must Be Isolated

You can deliberately inspect the invocation list and invoke each handler separately.

C#

```
EventHandler<GiftCompletedEventArgs>?
    handlers = GiftCompleted;

if (
    handlers is not null)
{
    foreach (
        EventHandler<GiftCompletedEventArgs> handler
        in handlers.GetInvocationList())
    {
        try
        {
            handler(
                this,
                args);
        }
        catch (
            Exception ex)
        {
            // Intentional policy:
            // log and continue.
        }
    }
}
```

This is an advanced policy decision, not the default behavior.

25. Handler Order

EVENTS / C#

Subscriber handlers are normally invoked in multicast delegate invocation-list order.

Do not build critical business correctness around:

```
EXAMPLE / FLOW
Handler A must always execute before B
```

unless the publisher explicitly owns that ordering contract.

If ordering is business-critical:

```
EXAMPLE / FLOW
Use explicit orchestration
```

instead of loosely coupled event subscribers.

26. Events Are Synchronous by Default

This code:

```
C#
GiftCompleted?.Invoke(
    this,
    args);

ContinueProcessing();
```

means:

```
EXAMPLE / FLOW
Raise event
    ↓
Execute all synchronous handlers
    ↓
Only then
    ↓
ContinueProcessing()
```

Events don't automatically create:

```
EXAMPLE / FLOW
Background work

Parallelism

Separate threads
```

Microsoft explicitly documents ordinary event subscribers as synchronously invoked.

27. Reentrancy

A subscriber can call back into the publisher while the publisher is still raising the event.

Example:

```
C#
private void HandleGiftCompleted(
    object? sender,
```

```
    GiftCompletedEventArgs e)
{
    if (
        sender is GiftProcessor processor)
    {
        processor.ProcessNextGift();
    }
}
```

EVENTS / C#

This can produce:

```
EXAMPLE / FLOW
Publisher operation
    ↓
Raises event
    ↓
Subscriber
    ↓
Calls publisher again
    ↓
Nested/reentrant operation
```

That can be dangerous if the publisher assumes no code can reenter it during event handling.

Avoid Raising Events While Holding Locks

Suppose:

```
C#
lock (_gate)
{
    UpdateState();

    GiftCompleted?.Invoke(
        this,
        args);
}
```

Subscriber code is external code from the publisher's perspective.

A handler might:

```
EXAMPLE / FLOW
Block

Throw

Acquire another lock

Call back into publisher
```

potentially creating:

```
EXAMPLE / FLOW
Deadlocks

Long lock durations

Reentrancy bugs
```

A safer pattern is often:

```
Lock
    ↓
Update internal state
    ↓
Release lock
    ↓
Raise notification
```

when your consistency requirements permit it.

28. Custom `add` and `remove` Accessors

Normally:

```C#
public event EventHandler?
    GiftChanged;
```

causes the compiler to provide event subscription accessors.

For specialized cases, you can implement:

```C#
add
```

and:

```C#
remove
```

yourself.

Microsoft notes that custom event accessors are uncommon; compiler-generated ones are sufficient for most events.

Example

```C#
private EventHandler?
    _giftChanged;

public event EventHandler GiftChanged
{
    add
    {
        _giftChanged += value;
    }

    remove
    {
        _giftChanged -= value;
    }
}
```

Raise the backing delegate:

```C#
_giftChanged?.Invoke(
    this,
    EventArgs.Empty);
```

Why Use Custom Accessors?

Possible advanced reasons include:

```EXAMPLE / FLOW
Centralized handler storage

Subscription logging

Interface-event mapping

Special synchronization

Memory optimization for many rarely used events
```

Don't use custom accessors simply because they look more advanced.

29. Events in Interfaces

Interfaces can declare events:

```C#
public interface IGiftProcessor
{
    event EventHandler<GiftCompletedEventArgs>
        GiftCompleted;

    void Process(
        Gift gift);
}
```

Implementation:

```C#
public sealed class GiftProcessor :
    IGiftProcessor
{
    public event EventHandler<GiftCompletedEventArgs>?
        GiftCompleted;

    public void Process(
        Gift gift)
    {
        // Work.

        GiftCompleted?.Invoke(
            this,
            new GiftCompletedEventArgs(
                gift.GiftId,
                gift.TotalAmount,
                DateTimeOffset.UtcNow));
    }
}
```

C# interfaces formally support event members, including explicit event implementations when multiple interfaces need different event behavior.

30. Static Events

Events can be:

```
C#
static
```

Example:

```
C#
public static event EventHandler?
    GlobalChanged;
```

Subscribers don't need a publisher instance.

But static events deserve caution.

Static Event Lifetime

A static publisher typically lives for approximately:

```
EXAMPLE / FLOW
Application / load-context lifetime
```

so:

```
EXAMPLE / FLOW
Static event
    ↓
Subscriber
```

can create especially long-lived references.

If a short-lived object subscribes and never unsubscribes, it can remain reachable much longer than intended.

Use static events only when the global lifetime is genuinely appropriate.

31. C# 14 Partial Events

C# 14 added:

```
C#
partial event
```

support.

A partial event has:

```
EXAMPLE / FLOW
Defining declaration
+
Implementing declaration
```

The defining side uses field-like event syntax, and the implementing side supplies `add` and `remove` accessors.

This is mainly useful for:

```
EXAMPLE / FLOW
Source-generated code

Split generated/manual implementations

Framework tooling
```

rather than everyday domain events.

Conceptual Example

One partial declaration:

```
C#
public partial event EventHandler?
    GiftCompleted;
```

The corresponding implementing part provides:

```
EXAMPLE / FLOW
add

remove
```

behavior.

For ordinary application events:

```
C#
public event EventHandler?
    GiftCompleted;
```

remains simpler.

32. Cancelable Events

Some events represent something that is:

```
EXAMPLE / FLOW
About to happen
```

and subscribers may need to request cancellation.

.NET provides:

```
C#
CancelEventArgs
```

for that style.

Microsoft's event-design guidance recommends considering cancelable pre-events where users should be able to prevent an action.

Example

C#

```
public sealed class GiftCancellingEventArgs :
    EventArgs
{
    public bool Cancel
    {
        get;
        set;
    }

    public string? Reason
    {
        get;
        set;
    }
}
```

Publisher:

C#

```
public event EventHandler<GiftCancellingEventArgs>?
    GiftCancelling;
```

Raise before mutation:

C#

```
GiftCancellingEventArgs args = new();

GiftCancelling?.Invoke(
    this,
    args);

if (
    args.Cancel)
{
    return;
}

CancelGift();
```

Pre-Event vs Post-Event

EXAMPLE / FLOW

```
GiftCancelling
    →
Operation has not happened yet
```

EXAMPLE / FLOW

```
GiftCancelled
    →
Operation already happened
```

The first may allow cancellation.

The second normally announces completed fact.

33. Observer Pattern

Events naturally support an observer-style design.

Traditional Observer roles:

EXAMPLE / FLOW

```
Subject
    →
Publisher
```

EXAMPLE / FLOW

```
Observer
    →
Subscriber
```

Flow:

EXAMPLE / FLOW

```
Publisher state changes
        ↓
Publisher notifies
        ↓
Observers react
```

C# events provide built-in language support for this general notification model.

Event-Based Observer

EXAMPLE / FLOW

```
GiftProcessor
    ↓
GiftCompleted
    ↓
 ┌──────────┬──────────┬──────────┐
 ↓          ↓          ↓
Audit    Receipt   Metrics
```

Publisher doesn't need to know each subscriber's concrete implementation.

34. Event vs Callback

A callback generally represents:

EXAMPLE / FLOW

```
One caller supplies behavior
for an operation
```

Example:

C#

```
ProcessGift(
    gift,
    onCompleted);
```

An event represents:

EXAMPLE / FLOW

```
Publisher announces occurrence
to zero or more subscribers
```

Callback

EXAMPLE / FLOW

```
Caller
    ↓
supplies delegate
    ↓
Operation
```

Event

EXAMPLE / FLOW

```
Subscribers
    ↓
register earlier
    ↓
Publisher raises later
```

Use callbacks for:

EXAMPLE / FLOW

```
Per-operation behavior
```

Use events for:

EXAMPLE / FLOW

```
Ongoing notification relationship
```

35. Event vs Direct Method Call

Direct:

C#

```
_receiptSender.SendReceipt(
    gift);
```

means:

EXAMPLE / FLOW

```
GiftProcessor intentionally depends
on receipt delivery
```

Event:

C#

```
GiftCompleted?.Invoke(
    this,
    args);
```

means:

EXAMPLE / FLOW

```
GiftProcessor announces completion

Subscribers decide how to react
```

EVENTS / C#

Neither is universally better.

Use Direct Calls When

The operation is:

EXAMPLE / FLOW

```
Required

Business-critical

Part of the workflow

Must succeed before operation succeeds
```

Example:

EXAMPLE / FLOW

```
Save Gift transaction
```

probably shouldn't be an optional event subscriber if gift persistence is essential.

Use Events When

The reaction is more naturally:

EXAMPLE / FLOW

```
Notification

Secondary reaction

Optional observer

Loosely coupled extension
```

36. Event vs Domain Event

These terms are often confused.

A C# language event:

C#

```
public event EventHandler<GiftCompletedEventArgs>?
    GiftCompleted;
```

is an in-process language/runtime mechanism.

A **domain event** is a modeling concept:

EXAMPLE / FLOW

```
Something important happened
in the business domain
```

Example:

EXAMPLE / FLOW

```
GiftCompleted
```

EVENTS / C#

A domain event might be represented as:

C#

```
public sealed record GiftCompletedDomainEvent(
    long GiftId,
    decimal Amount,
    DateTimeOffset OccurredAt);
```

and dispatched through:

EXAMPLE / FLOW

```
Mediator

Application dispatcher

Message system

Outbox
```

It does not have to use C# `event`.

37. Event vs Integration Event

An integration event communicates across:

EXAMPLE / FLOW

```
Services

Processes

Applications
```

For example:

EXAMPLE / FLOW

```
Fundraising API
    ↓
Message broker
    ↓
Receipt Service
```

A plain C# event:

EXAMPLE / FLOW

```
Exists only in this process
```

and provides no automatic:

EXAMPLE / FLOW

```
Persistence

Retry

Durability

Cross-process transport
```

```
Exactly-once guarantee
```

Therefore:

> **Do not use C# `event` as though it were a durable message broker.**

For production integration workflows, concepts such as:

```
EXAMPLE / FLOW
Outbox

Queue

Service Bus

Message broker
```

belong at the integration architecture layer.

Fundraising Example

C# event:

```
EXAMPLE / FLOW
Gift object changed
    ↓
Update in-memory observer
```

can be appropriate.

Integration event:

```
EXAMPLE / FLOW
Gift successfully persisted
    ↓
OutboxMessage created
    ↓
Message published
    ↓
External receipt system
```

requires durable infrastructure.

38. Events in ASP.NET Core

Events are useful inside .NET applications, but Web APIs need careful lifetime thinking.

Example:

```
EXAMPLE / FLOW
HTTP Request
    ↓
Scoped GiftService
    ↓
Scoped subscriber
```

If both disappear after the request:

EXAMPLE / FLOW

```
Subscription lifetime
is naturally short
```

Dangerous Lifetime Combination

EXAMPLE / FLOW

```
Singleton Publisher
    ↓
Scoped Subscriber
```

If the scoped subscriber subscribes to the singleton and does not unsubscribe:

EXAMPLE / FLOW

```
Singleton retains subscriber
    ↓
Request completes
    ↓
Subscriber may remain alive
```

This defeats the intended scoped lifetime.

Better Choices

Depending on the requirement:

EXAMPLE / FLOW

```
Explicit service orchestration

Scoped publisher + scoped subscribers

IDisposable subscription cleanup

Mediator/event dispatcher

Background queue

Durable messaging
```

may be more appropriate.

39. Async Event Handlers

The conventional .NET `EventHandler` shape returns:

EXAMPLE / FLOW

```
void
```

Therefore an asynchronous conventional event handler looks like:

C#

```
private async void HandleGiftCompleted(
    object? sender,
    GiftCompletedEventArgs e)
{
    await SendReceiptAsync(
```

```
        e.GiftId);
}
```

EVENTS / C#

`async void` is one of the few cases where C# guidance allows this pattern because conventional event handlers must return `void`. However, the publisher cannot await the handler, and exceptions from `async void` don't behave like normal `Task` exceptions.

Important Async Event Problem

Publisher:

```
C#
GiftCompleted?.Invoke(
    this,
    args);
```

may continue immediately after the asynchronous handler reaches its first incomplete `await`.

Therefore:

```
EXAMPLE / FLOW
Raise event
    ↓
async handler starts
    ↓
handler awaits
    ↓
publisher continues
    ↓
handler finishes later
```

The publisher has no ordinary way to:

```
EXAMPLE / FLOW
await all handlers
```

using standard `EventHandler`.

If Publisher Must Await Subscribers

Then a standard C# event may be the wrong abstraction.

You might deliberately design:

```
C#
Func<GiftCompletedEventArgs, Task>
```

callbacks or an explicit asynchronous dispatcher such as:

```
C#
Task PublishAsync(
    GiftCompletedEvent message,
    CancellationToken cancellationToken);
```

Now the caller can await completion.

This separates:

from:

40. Event Handler Exceptions

For synchronous events, handlers run inside the raising call.

Therefore a subscriber can affect the publisher through:


```
Exceptions

Latency

Reentrancy
```

A publisher should know whether subscribers are:


```
Trusted internal collaborators

Optional observers

Third-party extension code
```

and define a clear error policy.

41. EventArgs Should Carry Useful Data

Bad:

C#

```
public sealed class GiftEventArgs :
    EventArgs
{
    public GiftProcessor EntireApplication
    {
        get;
    }
}
```

This creates broad coupling.

Better:

C#

```
public sealed class GiftCompletedEventArgs :
    EventArgs
{
    public long GiftId
    {
        get;
    }

    public decimal Amount
```

```
    {
        get;
    }

    public DateTimeOffset CompletedAt
    {
        get;
    }
}
```

Expose the data subscribers genuinely need.

42. Prefer Immutable Event Data

Once:

```
EXAMPLE / FLOW
GiftCompleted
```

has happened, event data usually represents a fact.

Prefer:

```
C#
public long GiftId
{
    get;
}
```

rather than:

```
C#
public long GiftId
{
    get;
    set;
}
```

unless subscriber mutation is deliberately part of the protocol.

Why?

With multiple subscribers:

```
EXAMPLE / FLOW
Subscriber A
    ↓
modifies EventArgs
    ↓
Subscriber B
    ↓
sees modified values
```

That can create hidden ordering dependencies.

43. Security Considerations

An event is not a security boundary.

Suppose:

```C#
GiftCompleted?.Invoke(
    this,
    args);
```

The fact that a handler received an event does not prove:

```EXAMPLE / FLOW
Authorization

Tenant permission

Authentication
```

Subscribers still need correct security context where necessary.

Sensitive Event Data

Avoid placing:

```EXAMPLE / FLOW
Access tokens

Passwords

Payment secrets

Full sensitive donor data
```

into broadly published `EventArgs` unless every subscriber legitimately requires the information.

More event subscribers mean more code can observe the data.

Tenant Data

If your system is multi-tenant:

```C#
GiftCompletedEventArgs
```

may reasonably include:

```C#
TenantId
```

so handlers can maintain tenant context.

But subscribers must still enforce appropriate rules.

44. Performance Considerations

Raising an event with:

```EXAMPLE / FLOW
N subscribers
```

generally requires invoking:

EXAMPLE / FLOW

```
N handlers
```

so notification cost grows approximately with subscriber count.

For ordinary business applications this overhead is usually tiny compared with:

EXAMPLE / FLOW

```
Database access

HTTP calls

File I/O
```

The bigger performance risk is often what the handlers themselves do.

Slow Handler

EXAMPLE / FLOW

```
Publisher
    ↓
Slow subscriber
    ↓
Publisher blocked
```

Because ordinary handlers execute synchronously, a slow subscriber increases publisher latency.

45. Common Mistake — Public Delegate Instead of Event

Bad:

C#

```
public Action<Gift>?
    GiftCompleted;
```

when you intend publisher/subscriber semantics.

Better:

C#

```
public event Action<Gift>?
    GiftCompleted;
```

Common Mistake — Letting Consumers Raise Notifications

An event protects:

EXAMPLE / FLOW

```
Who decides that the event occurred?
```

That should normally be the publisher.

External code should not be able to falsely announce:

EXAMPLE / FLOW

```
Gift completed
```

when the gift wasn't completed.

Common Mistake — Never Unsubscribing

Especially dangerous when:

EXAMPLE / FLOW

```
Publisher lifetime
>
Subscriber lifetime
```

The publisher can keep the subscriber reachable.

Common Mistake — Anonymous Handler You Need to Remove

Bad:

C#

```
publisher.GiftCompleted += (_, e) => Handle(e);
```

if later unsubscription is required but you didn't retain the delegate.

Store the handler or use a named method.

Common Mistake — Assuming Events Are Asynchronous

They aren't.

Ordinary event handlers execute synchronously.

Common Mistake — Assuming All Subscribers Always Run

A synchronous subscriber exception can stop later invocation.

Define your error policy.

Common Mistake — Using Event for Required Workflow

Suppose successfully processing a gift absolutely requires:

EXAMPLE / FLOW

```
Write Gift to database

Write allocation rows
```

Don't hide essential steps as optional event subscribers.

Use explicit application workflow for required operations.

Common Mistake — Event for Cross-Service Communication

This:

C#

```
public event EventHandler?
    GiftCompleted;
```

doesn't notify another server or application instance.

C# events are in-process.

Use integration messaging when crossing process boundaries.

Common Mistake — Long-Running Handler

```C#
private void HandleGiftCompleted(...)
{
    Thread.Sleep(
        30_000);
}
```

blocks the publisher for 30 seconds.

Events don't automatically isolate work.

Common Mistake — `async void` Everywhere

`async void` is appropriate primarily when a conventional event signature requires `void`.

For ordinary application methods or callbacks, use:

```C#
Task

Task<T>

Func<Task>
```

so work can be awaited and exceptions observed.

Common Mistake — Mutable Shared EventArgs

If subscriber A changes:

```C#
e.Amount = 0m;
```

subscriber B may observe that changed value.

Prefer immutable event data for completed facts.

Common Mistake — Static Event Without Lifetime Plan

Static publisher:

EXAMPLE / FLOW

```
Lives a very long time
```

Short-lived subscribers:

EXAMPLE / FLOW

```
Can remain referenced indefinitely
```

Unsubscribe or choose another architecture.

Common Mistake — Raising Under a Lock

A subscriber is arbitrary code.

Calling it while holding internal synchronization can create:

EXAMPLE / FLOW

```
Deadlocks

Reentrancy

Unexpected latency
```

Common Mistake — Event Storm

Suppose changing one Gift triggers:

EXAMPLE / FLOW

```
GiftChanged

AllocationChanged

CampaignChanged

DashboardChanged

AnalyticsChanged
```

which trigger more events recursively.

You can accidentally create:

EXAMPLE / FLOW

```
Hard-to-follow execution chains
```

Use events deliberately rather than turning every property assignment into a global notification.

Best Practices

Prefer standard .NET event shapes for reusable APIs:

C#

```
EventHandler

EventHandler<TEventArgs>
```

Use meaningful event names:

EXAMPLE / FLOW

```
Completing

Completed

Changed

Added

Removed
```

Use:

EXAMPLE / FLOW

```
EventArgs
```

or:

EXAMPLE / FLOW

```
SomethingEventArgs
```

for event data.

Raise field-like events using:

C#

```
Event?.Invoke(
    this,
    args);
```

Keep event data focused and preferably immutable.

Use:

C#

```
protected virtual void OnEventName(...)
```

when an inheritable type needs a controlled event-raising hook.

Don't normally make the event itself virtual.

Subscribe with:

C#

```
+=
```

and unsubscribe with:

C#

```
-=
```

when publisher lifetime can outlive the subscriber.

Avoid anonymous subscription if you know you must unsubscribe later unless you retain the delegate instance.

Do not assume events are asynchronous.

Keep handlers fast unless synchronous latency is intentionally part of the contract.

Don't raise events while holding locks unless the consequences are fully understood.

Define how subscriber exceptions should affect the publisher.

Use `async void` only where the event pattern requires it and handle exceptions carefully.

If notification must be awaitable, design an explicit `Task`-based dispatcher instead.

Use C# events for in-process notifications, not durable distributed messaging.

Use direct calls when a reaction is required for correctness.

Use event-style notification when reactions are genuinely observers or optional extensions.

Be especially careful with:

EXAMPLE / FLOW

```
Singleton publisher
+
short-lived subscriber
```

in ASP.NET Core.

Practical Exercise 1 — Basic Event

Create:

```C#
public sealed class GiftProcessor
{
    public event EventHandler?
        ProcessingCompleted;

    public void Process()
    {
        ProcessingCompleted?.Invoke(
            this,
            EventArgs.Empty);
    }
}
```

Subscribe:

```C#
processor.ProcessingCompleted += HandleCompleted;
```

Practical Exercise 2 — Custom Event Data

Create:

```C#
public sealed class GiftCompletedEventArgs :
    EventArgs
{
    public GiftCompletedEventArgs(
        long giftId)
    {
        GiftId = giftId;
    }

    public long GiftId
    {
        get;
    }
}
```

Then:

```C#
public event EventHandler<GiftCompletedEventArgs>?
    GiftCompleted;
```

Practical Exercise 3 — Multiple Subscribers

```C#
processor.GiftCompleted += audit.Handle;

processor.GiftCompleted += receipt.Handle;

processor.GiftCompleted += analytics.Handle;
```

Question:

How many handlers execute when the event is raised?

EVENTS / C#

Answer:

EXAMPLE / FLOW

```
Three
```

provided none earlier in the invocation chain terminates notification by throwing an unhandled exception.

Practical Exercise 4 — Unsubscription

C#

```
processor.GiftCompleted -= receipt.Handle;
```

Now the receipt subscriber no longer receives future notifications.

Practical Exercise 5 — Lifetime

Suppose:

EXAMPLE / FLOW

```
Publisher = Singleton

Subscriber = Scoped
```

Should the scoped subscriber subscribe indefinitely?

Usually:

EXAMPLE / FLOW

```
No
```

because the singleton publisher may retain it past its intended request scope.

Practical Exercise 6 — Protected Raise Method

C#

```
public event EventHandler?
    Changed;

protected virtual void OnChanged()
{
    Changed?.Invoke(
        this,
        EventArgs.Empty);
}
```

Derived classes call:

C#

```
OnChanged();
```

instead of directly invoking the base event.

Practical Exercise 7 — Cancelable Event


```C#
GiftCancellingEventArgs args = new();

GiftCancelling?.Invoke(
    this,
    args);

if (
    args.Cancel)
{
    return;
}
```

This models:

```EXAMPLE / FLOW
Operation requested

Subscribers may cancel

Operation proceeds only if allowed
```

Practical Exercise 8 — Async Handler

Conventional event handler:

```C#
private async void HandleGiftCompleted(
    object? sender,
    GiftCompletedEventArgs e)
{
    try
    {
        await SendReceiptAsync(
            e.GiftId);
    }
    catch (
        Exception ex)
    {
        // Explicit error policy.
    }
}
```

Remember: the publisher cannot await this handler.

Practical Exercise 9 — Awaitable Alternative

```C#
public interface IGiftEventDispatcher
{
    Task PublishAsync(
        GiftCompletedEvent message,
        CancellationToken cancellationToken);
}
```

This may be a better design when:

Practical Exercise 10 — Lambda Unsubscription

Instead of:

C#
```
publisher.Changed += (_, _) => Console.WriteLine("Changed");
```

when removal is required:

C#
```
EventHandler handler = (_, _) => Console.WriteLine("Changed");

publisher.Changed += handler;

// Later:
publisher.Changed -= handler;
```

Practical Exercise 11 — Event vs Direct Call

Requirement:

EXAMPLE / FLOW
```
Gift cannot be considered complete
unless database save succeeds.
```

Use:

EXAMPLE / FLOW
```
Explicit save workflow
```

rather than an optional:

EXAMPLE / FLOW
```
GiftCompleted subscriber
```

for persistence correctness.

Practical Exercise 12 — Event vs Integration Event

Requirement:

EXAMPLE / FLOW
```
Another microservice
must eventually receive
GiftCompleted
even after process restart.
```

A C# event is insufficient.

Use:

EXAMPLE / FLOW
```
Durable message/outbox infrastructure
```

because the notification must survive process boundaries and failures.

Exactly 25 Interview Questions

1. What is an event in C#?

An event is a member based on a delegate type that lets a publisher notify subscribed handlers that something occurred.

Consumers can subscribe or unsubscribe, while the publisher controls event raising.

2. What is the relationship between delegates and events?

A delegate represents callable behavior.

An event wraps delegate-based subscription with additional encapsulation.

A public delegate field can potentially be replaced or invoked externally, while an event normally allows external consumers only to:

C#

```
+=

-=
```

3. What is a publisher?

The publisher is the object that:

EXAMPLE / FLOW

```
Declares the event

Decides when it occurred

Raises the event
```

4. What is a subscriber?

A subscriber attaches an event handler and reacts when the publisher raises the event.

One event may have multiple subscribers.

5. How do you subscribe and unsubscribe?

Subscribe:

C#

```
publisher.Changed += HandleChanged;
```

Unsubscribe:

C#

```
publisher.Changed -= HandleChanged;
```

6. What happens when an event has no subscribers?

A nullable field-like event is normally:

```
null
```

until at least one handler subscribes.

Using:

```
C#
Changed?.Invoke(
    this,
    EventArgs.Empty);
```

simply performs no invocation when there are no subscribers.

7. What is EventHandler?

`EventHandler` is the standard .NET delegate for events that don't need custom event data.

Its conventional handler shape includes:

```
EXAMPLE / FLOW
object sender

EventArgs e
```

8. What is EventHandler<TEventArgs>?

It is the standard generic event delegate for events that carry event-specific data.

The first parameter identifies the sender, and the second carries event data.

9. What is EventArgs.Empty?

It is the conventional event-data value used when an event doesn't need additional event information.

10. Why create a custom EventArgs class?

It lets the publisher send strongly typed information to subscribers.

For example:

```
C#
GiftCompletedEventArgs
```

might expose:

```
EXAMPLE / FLOW
GiftId

Amount

CompletedAt
```

Custom event argument classes conventionally derive from `EventArgs`.

11. What does the sender argument represent?

It normally represents the object that raised the event.

When raising:

```C#
Event?.Invoke(
    this,
    args);
```

the subscriber receives that publisher as `sender`.

12. Why use `?.Invoke` to raise an event?

It invokes the event only when there are subscribers and avoids manually checking for `null`.

This is the standard modern raising form shown in Microsoft's C# event examples.

13. Why use a protected `OnEventName` method?

For an inheritable publisher, it centralizes raising logic and gives derived classes a controlled way to participate in event notification.

Microsoft recommends this pattern because derived classes cannot directly invoke an event declared by a base class.

14. Are C# events synchronous or asynchronous?

Ordinary event invocation is synchronous.

When the publisher raises the event, subscribed handlers execute before normal control returns to the publisher.

15. What happens if an event handler throws?

With ordinary multicast invocation, the exception propagates out of event raising and later handlers are not normally executed.

The publisher must deliberately implement another policy if handlers should be isolated.

16. Why can event subscriptions cause memory leaks?

A publisher's event delegate contains references to subscriber handlers.

If a long-lived publisher still references a subscriber, that subscriber can remain reachable and cannot be garbage-collected.

17. When should you unsubscribe from an event?

You should strongly consider unsubscription when:

EXAMPLE / FLOW

```
Publisher can outlive subscriber
```

especially for:

EXAMPLE / FLOW

```
Static publishers

Singleton publishers

Long-lived UI/application services
```

18. Why are anonymous event handlers difficult to unsubscribe?

Writing:

C#

```
publisher.Changed += (_, _) => Handle();
```

EVENTS / C#

creates a delegate instance that you may not have available later for `-=`.

Store the delegate or use a named method if later removal is required.

19. Can events be declared in interfaces?

Yes.

An interface can define an event contract, and implementing classes provide its implementation. C# also supports explicit event implementation for interface scenarios.

20. What are custom `add` and `remove` event accessors?

They let a type control what happens during subscription and unsubscription.

They are advanced features; normal field-like events usually rely on compiler-generated accessors.

21. What are partial events in C# 14?

C# 14 allows an event declaration to be split between a defining partial declaration and an implementing declaration.

The implementation supplies the event's `add` and `remove` behavior.

22. Why can asynchronous event handlers be problematic?

Traditional event handlers return `void`.

An `async void` event handler can't be awaited by the publisher, and its exceptions don't follow ordinary task-based exception propagation.

`async void` is primarily appropriate for event-handler scenarios, while normal async APIs should return `Task` or `Task<T>`.

23. What is the difference between an event and a callback?

A callback is usually supplied directly for one operation:

EXAMPLE / FLOW

```
Caller → operation → callback
```

An event represents an ongoing publisher/subscriber relationship:

EXAMPLE / FLOW

```
Subscribers register
    ↓
Publisher later announces occurrence
```

24. What is the difference between a C# event and a domain/integration event?

A C# event is an in-process language mechanism.

A domain event is a business-modeling concept representing something meaningful that happened.

An integration event communicates across process/service boundaries and normally requires messaging infrastructure, durability, and retry strategies.

25. When should events be used in a real application?

Events are best when:

EXAMPLE / FLOW

```
One component owns an occurrence

Zero or more observers may react

The reactions are secondary or extensible

Loose in-process coupling is desirable
```

Use explicit method/service calls when downstream behavior is required for correctness, and use durable messaging rather than C# events for cross-process communication.

Quick Revision

Concept	Meaning
Publisher	Declares and raises event
Subscriber	Registers handler
`event`	Encapsulated delegate notification
`+=`	Subscribe
`-=`	Unsubscribe
`EventHandler`	Standard no-custom-data event delegate
`EventHandler<TEventArgs>`	Standard event delegate carrying data
`EventArgs.Empty`	No custom event data
`OnXxx`	Protected raising method convention
Multicast	Multiple subscribers
`?.Invoke`	Raise only when handlers exist
Custom `add/remove`	Specialized subscription logic
Static event	Type-wide, long-lived event
Partial event	C# 14 split event declaration/implementation
Closure handler	Handler capturing external state
`async void` handler	Conventional async event form with important limitations
Observer style	Publisher notifies interested subscribers
Domain event	Business concept, not necessarily C# `event`
Integration event	Cross-process messaging concept

Complete Fundraising Example

We will model:

EXAMPLE / FLOW

```
Gift completes
    ↓
GiftProcessor publishes
GiftCompleted
    ↓
Audit subscriber
+
Receipt subscriber
```

Event Data

C#

```
public sealed class GiftCompletedEventArgs :
    EventArgs
{
    public GiftCompletedEventArgs(
        long tenantId,
        long giftId,
        decimal amount,
        DateTimeOffset completedAt)
    {
        TenantId = tenantId;

        GiftId = giftId;

        Amount = amount;

        CompletedAt = completedAt;
    }

    public long TenantId
    {
        get;
    }

    public long GiftId
    {
        get;
    }

    public decimal Amount
    {
        get;
    }

    public DateTimeOffset CompletedAt
    {
        get;
    }
}
```

Publisher

C#

```
public sealed class GiftProcessor
{
    public event EventHandler<GiftCompletedEventArgs>?
        GiftCompleted;
```

```
    public void Complete(
        Gift gift)
    {
        ArgumentNullException.ThrowIfNull(
            gift);

        gift.Complete();

        OnGiftCompleted(
            new GiftCompletedEventArgs(
                gift.TenantId,
                gift.GiftId,
                gift.TotalAmount,
                DateTimeOffset.UtcNow));
    }

    private void OnGiftCompleted(
        GiftCompletedEventArgs e)
    {
        GiftCompleted?.Invoke(
            this,
            e);
    }
}
```

Because this class is sealed, the raise method doesn't need to be:

C#

```
protected virtual
```

If it were intended as a base type, that pattern would be appropriate.

Audit Subscriber

C#

```
public sealed class GiftAuditSubscriber
{
    public void HandleGiftCompleted(
        object? sender,
        GiftCompletedEventArgs e)
    {
        Console.WriteLine(
            $"AUDIT Tenant={e.TenantId}, "
            +
            $"Gift={e.GiftId}, "
            +
            $"Amount={e.Amount:C}");
    }
}
```

Receipt Subscriber

C#

```
public sealed class GiftReceiptSubscriber
{
    public void HandleGiftCompleted(
        object? sender,
        GiftCompletedEventArgs e)
    {
        Console.WriteLine(
            $"Receipt requested for Gift {e.GiftId}.");
```

```
    }
}
```

Subscription

C#

```
GiftProcessor processor = new();

GiftAuditSubscriber audit = new();

GiftReceiptSubscriber receipt = new();

processor.GiftCompleted += audit.HandleGiftCompleted;

processor.GiftCompleted += receipt.HandleGiftCompleted;
```

Execute

C#

```
Gift gift = new(giftId: 1001, tenantId: 10, totalAmount: 500m);

processor.Complete(
    gift);
```

Event Flow

EXAMPLE / FLOW

```
processor.Complete(gift)
        ↓
gift.Complete()
        ↓
Create GiftCompletedEventArgs
        ↓
OnGiftCompleted(...)
        ↓
GiftCompleted?.Invoke(...)
        ↓
  ┌─────────────────┴─────────────────┐
  ↓                                   ↓
AuditSubscriber             ReceiptSubscriber
  ↓                                   ↓
Write audit                 Request receipt
```

Unsubscribe

When no longer needed:

C#

```
processor.GiftCompleted -= audit.HandleGiftCompleted;

processor.GiftCompleted -= receipt.HandleGiftCompleted;
```

Disposable Subscriber Example

When publisher lifetime is longer:

```C#
public sealed class GiftAuditSubscription :
    IDisposable
{
    private readonly GiftProcessor
        _processor;

    private bool _disposed;

    public GiftAuditSubscription(
        GiftProcessor processor)
    {
        _processor = processor;

        _processor.GiftCompleted += HandleGiftCompleted;
    }

    private void HandleGiftCompleted(
        object? sender,
        GiftCompletedEventArgs e)
    {
        Console.WriteLine(
            $"Gift {e.GiftId} completed.");
    }

    public void Dispose()
    {
        if (
            _disposed)
        {
            return;
        }

        _processor.GiftCompleted -= HandleGiftCompleted;

        _disposed = true;
    }
}
```

This connects directly to Topic 19:

EXAMPLE / FLOW

```
Subscription lifetime
    ↓
IDisposable
    ↓
Unsubscribe
```

Where Events Fit in a Fundraising Architecture

Good In-Process Example

EXAMPLE / FLOW

```
Gift editor
    ↓
GiftChanged event
    ↓
Update local UI/view state
```

or:

EXAMPLE / FLOW

```
In-memory workflow component
    ↓
StateChanged
    ↓
Observers
```

EVENTS / C#

Required Application Workflow

If successful completion requires:

EXAMPLE / FLOW

```
Persist Gift

Persist GiftAllocation

Commit SQL transaction
```

use explicit application orchestration:

EXAMPLE / FLOW

```
GiftService
    ↓
Repository / DbContext
    ↓
Transaction
```

Do not rely on optional C# event subscribers for transactional correctness.

Integration Workflow

Suppose completion must notify another system:

EXAMPLE / FLOW

```
Gift saved
    ↓
OutboxMessage stored
    ↓
Transaction commits
    ↓
Background publisher
    ↓
Message broker
    ↓
Receipt / Analytics system
```

That is a durable integration workflow.

A C# event alone cannot provide those delivery guarantees.

Final Event Decision Flow

EXAMPLE / FLOW

```
Something happened
        ↓
Is another component required
for correctness?
```

Yes:

EXAMPLE / FLOW

Explicit method/service workflow

EVENTS / C#

No:

EXAMPLE / FLOW

Are zero or more in-process
observers interested?

Yes:

EXAMPLE / FLOW

C# event may fit

Then ask:

EXAMPLE / FLOW

Does notification need custom data?

No:

C#

```
EventHandler
```

Yes:

C#

```
EventHandler<TEventArgs>
```

Lifetime Decision Flow

EXAMPLE / FLOW

```
Subscriber attaches
        ↓
Will publisher possibly outlive subscriber?
```

No:

EXAMPLE / FLOW

Natural lifetime may be sufficient

Yes:

EXAMPLE / FLOW

```
Plan unsubscription
    ↓
-= handler

or

IDisposable subscription lifetime
```

Async Decision Flow

EXAMPLE / FLOW

```
Notification handler needs async work?
        ↓
Does publisher need to wait
for completion?
```

No:

EXAMPLE / FLOW

```
Conventional async event handler
may be acceptable,
with explicit exception handling
```

Yes:

EXAMPLE / FLOW

```
Use Task-based
async dispatcher/callback abstraction
instead of ordinary EventHandler
```

Distributed Decision Flow

EXAMPLE / FLOW

```
Must subscriber run
in another process/server?
        ↓
Yes
```

Then:

EXAMPLE / FLOW

```
C# event is insufficient
        ↓
Use durable integration messaging
```

Final End-to-End Mental Model

EXAMPLE / FLOW

```
Delegate
    ↓
Represents callable behavior
```

EXAMPLE / FLOW

```
event
    ↓
Restricts delegate interaction
to publisher/subscriber model
```

EXAMPLE / FLOW

```
Publisher
    ↓
Owns occurrence
```

EXAMPLE / FLOW

```
Subscriber
    ↓
Registers reaction
```

EXAMPLE / FLOW

```
EventHandler<TEventArgs>
    ↓
Standard strongly typed
notification pattern
```

EXAMPLE / FLOW

```
+=
    ↓
Subscribe
```

EXAMPLE / FLOW

```
-=
    ↓
Unsubscribe
```

EXAMPLE / FLOW

```
?.Invoke
    ↓
Raise notification
```

EXAMPLE / FLOW

```
Long-lived Publisher
+
Short-lived Subscriber
    ↓
Potential lifetime leak
```

EXAMPLE / FLOW

```
C# event
    ↓
In-process synchronous notification
```

EXAMPLE / FLOW

```
Integration event
    ↓
Durable cross-process messaging
```

The most important lesson is:

A C# event is an encapsulated delegate-based notification mechanism. The publisher controls when the event is raised, while subscribers independently decide how to react. Use `EventHandler` or `EventHandler<TEventArgs>` for conventional .NET events, keep event data focused, unsubscribe when publisher lifetime can exceed subscriber lifetime, understand that normal handlers run synchronously, and do not confuse in-process C# events with durable domain or integration messaging.

For the Fundraising Platform, a practical default is:

EXAMPLE / FLOW

```
Small in-process observer notification
    →
C# event

Required business workflow
    →
Explicit service call

Rich async in-process dispatch
    →
Task-based dispatcher

Cross-service / durable notification
    →
Outbox + messaging infrastructure
```

Next roadmap topic: Topic 24 — LINQ.